

RESEARCH

Open Access



An assessment of large language models for OpenMP-based code parallelization: a user perspective

Marko Mišić^{1*} and Matija Dodović¹

*Correspondence:
marko.misic@etf.bg.ac.rs

¹ University of Belgrade, School of Electrical Engineering, Bulevar kralja Aleksandra 73, Belgrade 11000, Serbia

Abstract

Large language models have sparked a lot of attention in the research community in recent days, especially with the introduction of practical tools such as ChatGPT and Github Copilot. Their ability to solve complex programming tasks was also shown in several studies and commercial solutions increasing the interest in using them for software development in different fields. High performance computing is one of such fields, where parallel programming techniques have been extensively used to utilize raw computing power available in contemporary multicore and manycore processors. In this paper, we perform an evaluation of the ChatGPT and Github Copilot tools for OpenMP-based code parallelization using a proposed methodology. We used nine different benchmark applications which represent typical parallel programming workloads and compared their OpenMP-based parallel solutions produced manually and using ChatGPT and Github Copilot in terms of obtained speedup, applied optimizations, and quality of the solution. ChatGPT 3.5 and Github Copilot installed with Visual Studio Code 1.88 were used. We concluded that both tools can produce correct parallel code in most cases. However, performance-wise, ChatGPT can match manually produced and optimized parallel code only in simpler cases, as it lacks a deeper understanding of the code and the context. The results are much better with Github Copilot, where much less effort is needed to obtain correct and performant parallel solution.

Keywords: ChatGPT, Github Copilot, High performance computing, Large language models, OpenMP, Parallel programming

Introduction

Large language models (LLMs) have been constantly changing the landscape of artificial intelligence and natural language processing in recent years impacting different fields of life [1]. Based on probabilistic models and deep learning Transformer architecture [2], they can learn distinctive features from unstructured data without human intervention. Moreover, they are trained over a large quantity of data from the internet leading to models with billions of parameters and suitable for various applications.

There are numerous examples of successful application of LLMs in both research and commercial domains. They have been used for chatbots and other autonomous agents in customer support and marketing [3], finance [4], social and natural sciences [5],

biomedicine, genomics and healthcare [6, 7], translation and localization [8], content generation, education [9]. A more comprehensive survey of applications and guidance for practitioners working with LLMs can be found in refs. [10, 11].

Code generation was one of the focal applications of LLMs from the early days of their adoption. Codex [12] is just one of the notable examples, as a distinct production version of the Codex engine is employed in Github Copilot. The other notable examples of general models are GPT-3 [13] used in the OpenAI ChatGPT tool, DeepMind AlphaCode [14], and Code Llama [15]. Various domain-specific models have been also proposed to improve code understanding [16], code review [17], repair and refactoring [18], and parallel programming [19], as examples. Several authors showed the suitability of both ChatGPT and Github Copilot for daily programming tasks [12, 20], as well as programming assignments in educational context [21]. Moreover, a serious debate among educators arose on how to adapt to the utilization of those tools in the domain of computing education, as more and more students use them in their academic work [22].

In this paper, we propose a novel methodology to evaluate the usability and applicability of ChatGPT and Github Copilot in the context of shared memory parallel programming on the multicore central processing units (CPUs). We focused our research on the OpenMP programming model and used both tools to annotate diverse applications from the high-performance computing domain (HPC) with OpenMP directives and clauses.

We summarize the contributions of our work as follows:

- We proposed a novel experimental methodology to evaluate LLM-based tools for OpenMP-based parallelization tasks,
- We formed a benchmark suite comprising of nine different compute-bound and memory-bound real-life mini-applications for evaluation of LLM-based tools for OpenMP-based parallelization tasks,
- We conducted an experiment and evaluated ChatGPT and Github Copilot tools for OpenMP-based parallelization tasks using the proposed benchmark suite and compared the results against manually parallelized code.

The paper is divided as follows, following the general structure proposed in refs. [23]. In the Sect. 2 we define in more detail the problem tackled in the paper. Related work is presented in the Sect. 3. In Sect. 4, we describe benchmark applications used in our experiments followed by the design of the experiment. The Sect. 5 elaborates the results together with the discussion on experiences gained through the process. The final section briefly concludes the paper with directions for future work.

Problem definition

Modern central processing units (CPUs) and graphics processing units (GPUs) usually consist of dozens of cores. However, to exploit parallelism available in hardware, we need to write parallel programs. Rewriting existing sequential programs and writing inherent parallel programs is a challenging task even for an experienced software developer, as one should care about finding parallelism in the problem being solved, the programming model used, data sharing, synchronization, and communication between parallel tasks, and similar. It is even more emphasized in high-performance

computing (HPC) settings where large-scale parallelism should be exploited [24] for big data, simulation, and machine learning purposes [25–27].

For that reason, numerous tools and programming models have been used to ease the process of writing parallel programs and to enable reaching the best out of available hardware systems [28]. One such well-established shared memory programming model is OpenMP [29] which is used for directive-based parallelization of sequential code on CPUs and GPUs. Usually, the programmer annotates loops in code with a set of directives which are in turn used to automatically parallelize them by the compiler. They should carefully decide which constructs to use, as inappropriate usage of the directives and clauses can lead to bugs and performance degradation. Numerous tools have been developed to assist programmers in discovering parallelism or transforming sequential code into OpenMP-annotated code for particular domains. For example, AutoPar-Clava [30] statically detects loops that can be parallelized and data access patterns, and sub-sequentially automatically annotates the code. SPar [31] automatically generates code targeting OpenMP runtime for parallel stream processing applications. However, as pointed out in [32], such tools tend to be conservative, and they sometimes miss potential parallelization opportunities.

With the advancements in natural language processing and LLMs trained on code, several new approaches based on Transformer architectures emerged to address the problem of directive-based and automated code parallelization, such as OMPify [33], AutoParLLM [32], and OMPGPT [34]. Both OMPify and AutoParLLM combine the power of graph neural networks (GNNs) with Transformer models to produce OpenMP parallelized code. OMPGPT further enhances the effectiveness of the models with considerably smaller size and chain-of-thought prompt engineering, making it suitable for usage in typical hardware settings of HPC environments. It should be noted that LLM applications have also been proposed in other HPC domains, such as code analysis, race detection, performance optimization, and question answering. HPC-specific LLMs, pipelines, and workflows have been proposed in the open literature for such tasks [35–37]. However, all aforementioned models and tools present valuable academic efforts, but still are not readily available to wider public for everyday usage.

In this paper, we adopted a more user-centered approach to explore the usability of LLMs for OpenMP-based parallel programming. Our idea was to assess existing, widely and readily available LLM-based tools as a practical aid to programmers developing parallel programs and to define a set of guidelines which can help programmers to use existing LLM tools in more efficient way for such a specific purpose. In the context of 10 methods for induction of new research ideas presented in the paper [38], our effort can be seen as a specialization: starting from a well-established general approach and tools, we stem to derive a specific knowledge for a specific domain, parallel programming. We used nine different benchmark applications from various HPC datasets which represent typical parallel programming workloads and compared their OpenMP-based parallel solutions produced manually and using ChatGPT and Github Copilot, as widely available LLM-based tools, in terms of obtained speedup, applied optimizations, and quality of the solution. We present our experiences and efforts needed to obtain correct and performant parallel programs.

Existing solutions

Large language models are probabilistic models of natural languages implemented using neural networks and deep learning techniques and trained on massive datasets of text and code [39]. During the training process, probabilistic analysis of unstructured data is performed to learn deep, non-trivial relationships in data. The LLM training data is predominantly collected from the Internet, which allows models to generate diverse content that resembles content generated by humans. In a short period of time LLMs have proven themselves in natural language understanding, content generation, knowledge-intensive tasks, reasoning ability, handling noisy input, unformalized tasks, and human instructions [10].

Large language models trained for code first showed promising results for tasks such as code completion [40, 41] and generation from natural language descriptions [12]. Several early studies focused on code generation for Python programming language based on documentation strings or problem descriptions given in prompts [12, 42]. However, other languages are also taken into consideration. The study [43] presents a systematic evaluation of five LLMs using 12 different programming languages where their model, PolyCoder outperformed other models for C programming-related tasks. The other comprehensive study focused on AI-generated code for web development [44]. It showed that both ChatGPT and Copilot produced code in accordance with the predefined requirements and similar in quality.

LLMs have been utilized for parallel programming, especially for code completion, OpenMP annotation, and performance modeling tasks with varying success [45]. Performance modeling is especially pronounced in this context, as performance data (execution time) based on source code is usually not available. On the other hand, several models specifically tuned for OpenMP annotations emerged in the open literature, such as OMPify [33], AutoParLLM [32], and OMPGPT [34]. OMPify is based on a Transformer-based model that leverages a graph-based representation of source code that exploits the inherent structure of code. AutoParLLM is a framework for automatically discovering parallelism and generating the parallel version of the sequentially written program based on heterogeneous graph neural network and LLM-based code generator. OMPGPT uses GPT-Neo 2.7B, pre-trained using C and C++ HPC code. Its design is based on chain-of-OMP, a variant of chain-of-thought prompting [46] to LLMs, an innovative strategy designed to enhance LLM effectiveness for OpenMP-based parallelization.

Proposed models, such as OMPify, AutoParLLM, and OMPGPT are often evaluated on different HPC datasets. For example, OMPify is evaluated on the applications from commonly used parallel benchmark sets, such as NAS, SPEC, and PolyBench. AutoParLLM uses applications from NAS Parallel Benchmark and Rodinia Benchmark, while OMPGPT uses newly proposed HPCorpus [47] based on GitHub repositories of C, C++, and Fortran code. A novel suite, PCGBench was proposed in [19], consisting of a set of 420 tasks for evaluating the ability of language models to generate parallel code. However, the aforementioned tools still present efforts from academia that are primarily aimed at improving LLMs and related workflows for HPC using different metrics primarily intended to evaluate LLM capabilities. Moreover, most of those tools are not readily available, nor actively used for parallel programming.

ChatGPT was evaluated for parallel programming with OpenACC [48] in a similar user-centric fashion to our approach. OpenACC is a directive-based programming model, similar to OpenMP, primarily intended for GPUs. The authors performed a series of experiments using four different problems: calculation of Mandelbrot fractal, matrix multiplication, Jacobi method, and primer numbers. However, they did not use existing applications, but asked ChatGPT to produce both sequential and OpenACC-annotated code. They compared the results to the manually written code with mixed results. Their conclusion was that human-generated codes were more efficient in all cases, as humans can employ OpenACC directives and clauses more effectively, although the ChatGPT version of the Jacobi method was very close performance-wise.

Proposed solution

In this section, we discuss our proposed methodology and experiments. First, we discuss used benchmark applications and the reasons for their selection. Then, we describe in detail our experimental setup and design.

Benchmark applications

Benchmarking of HPC systems based on CPUs and GPUs and corresponding programming models has attracted a lot of research interest in the past. Benchmark suites usually consist of carefully chosen mini-applications representing various real-life workloads and computational patterns. Usually, they consist of sequential, baseline implementations together with differently sized input and correct output data and corresponding testing framework. Well-known benchmark suites include SPEC [49], NAS [50], Poly-Bench [51], Rodinia [52], and Parboil [53]. Reference implementations for OpenMP, CUDA, OpenCL, OpenACC, and other relevant programming models are also present in some of them, depending on the benchmark support for execution on the CPUs and GPUs. New implementations for different platforms and programming models have been constantly developed [54] [55].

In our work, we did not resort to any particular benchmark suite, as they are usually tailored for specific scenarios and use cases. Instead, we have chosen nine mini-applications from various sources to particularly address the problems for OpenMP parallelization in different commercial and scientific fields, mainly from the domains such as numerical methods and physics simulation. They come from the following sources, respectively: John Burkardt's repository of code from the Florida State University [56], Rodinia benchmark suite [52], Edinburgh Parallel Computing Centre (EPCC) training examples [57], an Introduction to Parallel Programming (ITPP) book [58], AMD HPC training examples [59], and Parboil benchmark suite [53]. The complete list of chosen mini-applications together with their short descriptions are given in Table 1. The suite includes both compute-bound and memory-bound applications and they were primarily chosen to test different OpenMP features in their solutions at various levels of difficulty. Those features include *reduction* clause, different scheduling strategies due to loop imbalance, atomic updates, loop fusion through *collapse* clause, etc.

To additionally characterize the selected mini-applications, we followed a well-known dwarf taxonomy from Berkeley [60], also utilized in assembling [52] and characterizing [61] Rodinia benchmark suite. We also characterized mini-applications by simpler

Table 1 A list of mini-applications used in assessment of LLMs for OpenMP-based parallelization

Application	Source	Type	Description
Feynman	Burkardt	Compute bound	Feynman-Kac algorithm to solve Poisson's equation in 3D interval
HotSpot	Rodinia	Memory bound	Thermal simulation
Mandelbrot	EPCC Training Examples	Compute bound	Calculates an area of the Mandelbrot set
MolDyn	EPCC Training Examples	Compute bound	Molecular dynamics simulation
Nbody	ITPP Book	Compute bound	N-body interaction simulation
PiCalculation	ITPP Book	Compute bound	Estimates the value of π
Prime	Burkardt	Compute bound	Calculates prime numbers
Saxpy	AMD HPC Training Examples	Memory bound	Scalar multiplication and vector addition operation
Sgemm	Parboil	Memory bound	Single precision dense matrix-matrix multiplication kernel

Table 2 Characterization of mini-applications used in the assessment of LLMs for OpenMP-based parallelization

Application	Dwarf	Domain	Parallel programming patterns
Feynman	Unstructured grid	Physics, statistics, finance, chemistry	Reduction, loop imbalance
HotSpot	Structured grid	Physics simulation	Stencil computation, loop fusion
Mandelbrot	Monte Carlo	Complex dynamics, fractal geometry	Reduction, loop imbalance
MolDyn	N-Body methods	Molecular dynamics	Reduction, loop imbalance, atomic update
Nbody	N-Body methods	Physics simulation	Reduction, loop fusion
PiCalculation	Monte Carlo	Numerical method	Reduction
Prime	Structured grid	Primality testing	Search, reduction, loop imbalance
Saxpy	Dense linear algebra	Numerical method	Array computations, transform
Sgemm	Dense linear algebra	Numerical method	Matrix computations, loop fusion

problem types and parallel programming patterns, similar to those presented in ref. [19]. The summary is shown in Table 2. It can be seen that our choice is dominated by the mini-applications from the domain of numerical methods and physics simulations, which are frequent targets for parallelization and use case of parallel programming and HPC systems. We also envisioned the inclusion of graph traversal applications, such as breadth search first (BFS) algorithm. However, due to our previous experience [55], we intentionally omitted it in the current research, as it performs poorly in parallel environments.

Feynman is an application that uses the Feynman-Kac algorithm to solve Poisson's equation in a 3D interval by averaging stochastic paths to the boundary. It is a compute-bound application where the number of iterations directly affects the precision of the results. The baseline application is interesting, as it exhibits a fairly low level of parallelism in the outer loops. *HotSpot* is a widely used thermal simulation to estimate chip temperature over a finite grid quadratic surface. It iteratively solves a series of differential equations on a grid of cells where each cell represents the average temperature value of the corresponding area of the chip.

Mandelbrot program approximately calculates the area of the Mandelbrot set which is a set of complex numbers satisfying certain criteria. Essentially, it is a Monte Carlo

simulation that iterates over a set of points in grid space defined by the complex plane. Each point is iterated for a defined number of iterations to check if it belongs to the Mandelbrot set. The number of points outside the set is accumulated, and based on that, an approximation of the area is calculated.

MolDyn is a molecular dynamics simulation of atoms in a box with periodic boundary conditions. The particles (atoms) are arranged in a lattice with initialized forces and velocities. In each iteration, particles are moved depending on their velocities and forces, which are in turn calculated together with kinetic and potential energies for the subsequent iterations. The correct parallel solution needs synchronization constructs at several points in the code.

Nbody represents a simple gravitational n-body simulation. It implements a 2D n-body solver that uses the reduced algorithm. *PiCalculation* estimates the value of pi using n terms of the Maclaurin series. The solution requires a reduction over the final sum. *Prime* counts the number of primes between 1 and n using a rather naïve, inefficient algorithm. A `reduction` clause is needed to correctly sum the number of primes.

Saxpy implements scalar multiplication and vector addition operations. *Sgemm* is a dense linear algebra single precision general matrix multiplication operation defined as a part of BLAS (Basic Linear Algebra Subprograms) collection and a frequently used operation in numerical linear algebra applications.

Experimental setup and design

Before the experiment, all mini-applications were carefully inspected and slightly re-implemented. They were all adapted to use the same build system (*Makefile*). The used compiler was *gcc 9.3.0* with compilation flags *-std=gnu99 -fOpenMP -O2 -lm*. Timers from the OpenMP library were introduced to all applications in order to measure time in a uniform manner. All baseline versions were tested against known gold standards using defined input data. Where available, different input data sizes were used to test the scalability of the solutions. All tests were conducted on the machine with Intel Core i7-11700F CPU 2.50GHz with 8 physical cores per socket, 16 virtual cores and 64GB RAM using Ubuntu 22.04 LTS. ChatGPT 3.5 was used. GitHub Copilot was used with Visual Studio Code, version 1.88 from March 2024. All solutions were tested using 8 threads.

The experiment is conducted with five different versions of the mini-applications. The baseline version (*base*) is used to measure the time of the sequential execution and ensure the correctness of parallel implementations. Four more OpenMP versions were produced. The first version (hereinafter *omp*) presents a best effort by the authors of the paper to parallelize the application suite manually using all the skills and knowledge that the programmer possesses from the domain of OpenMP programming. Some of the mini-applications had their OpenMP implementations provided. Those were thoroughly inspected and improved by the authors of the papers, if needed. The others were parallelized from their baseline versions. However, the optimizations performed by the authors of the paper included only those available through OpenMP library. Some manual optimizations, such as variable privatization were intentionally omitted, as they are usually left to the decision by the compiler [62].

The experimentation further extends to include two variations facilitated by the ChatGPT tool with the series of interactive conversations, labeled *omp_gpt* and *omp_gpt2*, as well as a version produced by GitHub Copilot extension, referred to as the *copilot* version. In *omp_gpt* version, ChatGPT was given just with the code snippet that was identified by the programmer as a potential source of parallelism. Those snippets of code are usually loops, as loop iterations are usual source of parallelism in sequential code. Those loops do not necessarily represent the whole function bodies, as they can be rather long or contain several parallelizable loops. In *omp_gpt2* version, each mini-application was given to the ChatGPT with the whole context (whole code of the application) which can help the tool to parallelize it by giving additional insights, as suggested in ref. [32]. The initial prompt for both ChatGPT variants was consistent: “Try to parallelize the code snippet given below using OpenMP?” followed by the appropriate piece of code.

After each answer from ChatGPT that produced the piece of code, it was integrated into the mini-application, compiled and tested with appropriate examples. If the code did not compile, a number of compilation attempts were incremented, and the interactive process with ChatGPT continued providing hints of what went wrong, e.g. by pasting compiler errors or similar. Once compiled successfully, the next stage assessed the correctness of the code’s execution by comparing the results to the baseline version. If the output was incorrect, the user would prompt ChatGPT with the observed differences between the expected and incorrect results, and if necessary, guided the ChatGPT with suggestions to achieve correct results. These prompts contributed to the number of correct-execution-prompts.

After successful compilation and correct execution, ChatGPT was asked to optimize the code with the following question “Can you further optimize the loop using OpenMP directives and clauses?”. Depending on the answers, and status of compilation, this process was repeated until satisfactory results in terms of performance or quality of the solution were met. From a performance viewpoint, the process was stopped when the results were similar or exceeded the results of the *omp* version. As for the quality of the solution, the process was stopped when all relevant OpenMP constructs that can be applied in the solution for the problem were explored by the ChatGPT after hints from the prompt. This process of iterative development for both versions, *omp_gpt* and *omp_gpt2*, guided by specific counters at each stage of compilation success, correct execution, and performance optimization, was visualized in the (a) diagram of Fig. 1.

The experiment design using GitHub Copilot for the parallelization of code requires a systematic approach to identify all hotspots that can benefit from parallelization. For that reason, the initial prompt in our study for GitHub Copilot experiment is different than in the case of ChatGPT. It combines several different prompts at several places in the identified code snippet. The primary focus is on locating loops suitable for OpenMP annotation using `#pragma omp for` directives and sections in those loops where synchronization may be necessary by using `#pragma omp atomic` or similar constructs. Although task-based parallelism is possible, which was suggested by the tool itself in the case of Hotspot mini-application, empirical evidence suggests Copilot frequently opts for loop-based parallelization. However, we assume that omission of explicit task-based parallelism is not a significant limitation of this study. Explicit task-based parallelization is usually employed when loop-based parallelization strategies cannot be employed (i.e.

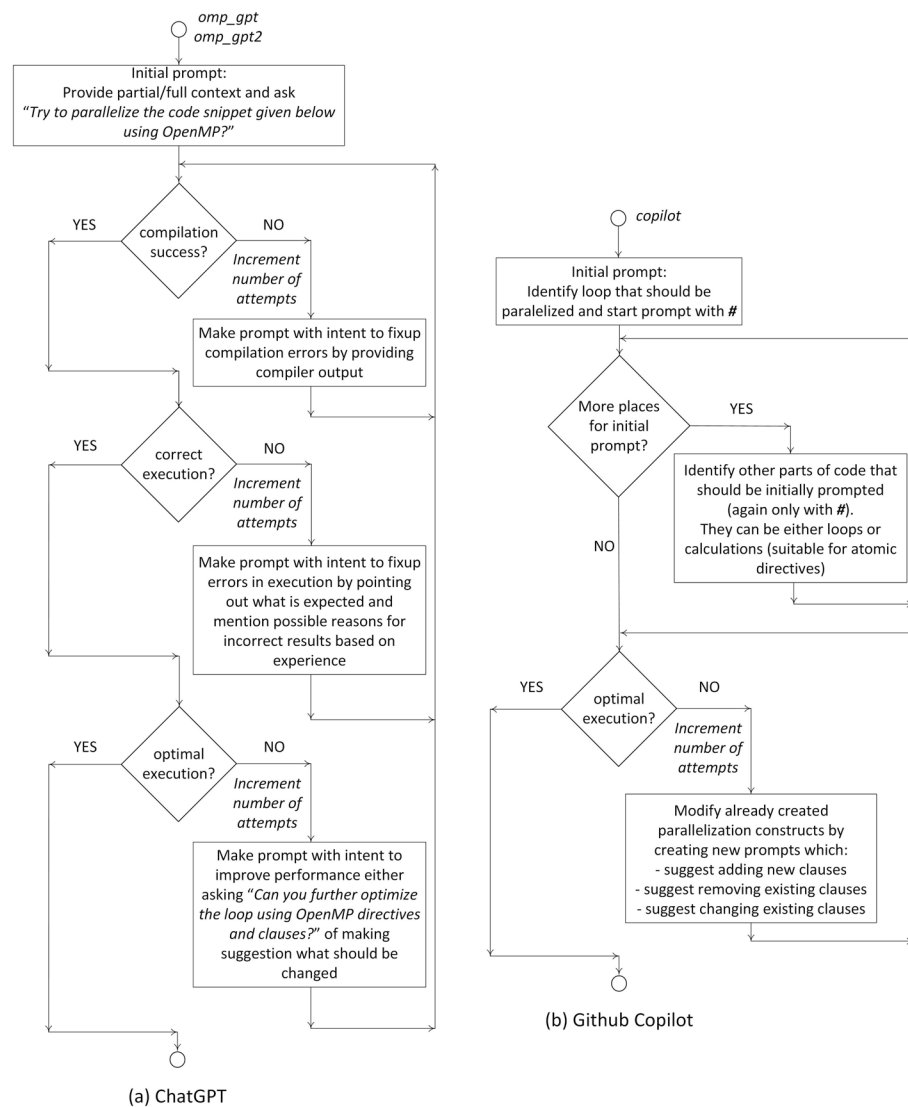


Fig. 1 Diagram of the experiment for **a** ChatGPT's variants: *omp_gpt* and *omp_gpt2* and **b** Github Copilot variant: *copilot*

when number of iterations is not formally decidable in the compile time). Those situations are not so common in OpenMP applications and not present in the examples of our benchmark suite. They form an interesting avenue for future work with both ChatGPT and GitHub Copilot tools.

The experiment starts with an initial prompt starting with a #, inviting Copilot to suggest parallel constructs. This initial prompt is applied universally (only #) across all potential parallelization locations manually identified in the code, as Github Copilot can insert code in prompted place only. Copilot's output at this stage is considered the combined parallelization effort of all such sites. The tool's capacity to consider the complete context of the file enables it to produce a block of code that does not violate successful compilation and correctness of execution, after initial prompting is finished. If the parallelized code performance does not yield optimal execution, the cycle enters an iterative

phase. A counter tracks the number of prompts, each tailored to refine the parallelization. The process is meticulous: one specific location in code is chosen for enhancement, and the prompt is designed to either add, remove, or change clauses in the parallel construct at that site. The modified code is executed and evaluated for performance gains or losses.

The reason for not counting the number of prompts necessary for successful compilation or correct execution is rooted in Copilot's design, which inherently analyzes the full file context and specializes in generating operational code on the fly [63]. This differs from the approaches necessary for *omp_gpt* and *omp_gpt2* versions, where the number of prompts may significantly impact compilation and correctness. In this experiment, the singular metric of interest is the improvement of performance. The outcome of this experimental design is visually summarized in the (b) diagram of Fig. 1.

Elaboration

In this section, we present the results of our experiments and discuss them in detail. Table 3 shows the statistics from the ChatGPT interaction: number of attempts (queries) for each mini-application before first compilation success, first correct execution, and optimal execution for *omp_gpt* and *omp_gpt2* versions. It also shows the number of attempts needed for GitHub Copilot to produce the optimal version. In the case of ChatGPT, it can be noticed that only PiCalculation and Saxpy applications were parallelized and optimized smoothly without issues. In all other cases, further interaction with the user was needed.

Feynman

In parallelizing the Feynman algorithm, ChatGPT adeptly recognized and optimized three nested loops by reallocating a segment of code to the third loop, utilizing the collapse function to enhance execution efficiency. However, the *omp_gpt* variant did not include the `schedule(dynamic)` directive, a decision that was rectified in the *omp_gpt2* version. This adjustment, informed by a broader contextual understanding, allowed *omp_gpt2* to dynamically distribute iterations, optimizing workload distribution and

Table 3 Number of prompts for each mini-application before first compilation success, first correct execution for *omp_gpt* and *omp_gpt2* for ChatGPT and optimal execution for all three LLM-based versions (*omp_gpt*, *omp_gpt2*, and *copilot*)

Application	Compilation success		Correct execution		Optimal execution		
	<i>omp_gpt</i>	<i>omp_gpt2</i>	<i>omp_gpt</i>	<i>omp_gpt2</i>	<i>omp_gpt</i>	<i>omp_gpt2</i>	<i>copilot</i>
Feynman	1	1	1	1	2	1	1
HotSpot	1	1	2	1	2	3	3
Mandelbrot	2	1	2	1	7	3	1
MolDyn	1	3	2	6	2	8	3
Nbody	2	1	3	2	8	7	3
PiCalculation	1	1	1	1	1	1	1
Prime	1	1	1	1	3	6	1
Saxpy	1	1	1	1	1	1	1
Sgemm	1	1	2	1	3	2	2

improving parallel performance. GitHub Copilot, from the outset, accurately introduced single `#pragma omp for` directive without the collapse clause, hence it has no capabilities of reorganizing code.

Additionally, with the *omp_gpt2* variant, ChatGPT pinpointed the segment where the seed for random number generation was set, modifying the code to assign each thread its own local seed, ensuring thread-safe and consistent random number generation across threads. Figure 2 presents the speedup achieved using different prompts for the parallelization of the Feynman algorithm. It also clearly illustrates the consequences of excluding the `schedule(dynamic)` directive in the *omp_gpt* variant, resulting in a noticeable performance decrease when compared to the *omp_gpt2* variant, which included the directive, and the code produced by GitHub Copilot. It can be noticed in Fig. 2 that *omp_gpt2* and *copilot* versions achieved larger speedup than manually produced *omp* version. The reason stemmed from the utilization of `private` clause for several variables which resulted in their privatization (replication) among the threads and possible allocation in registers. However, we intentionally omitted such optimization from the manually produced *omp* version, as it cannot be seen as a purely OpenMP feature, but rather compiler optimization. In this case, it helped to achieve higher speedup. However, in some other cases, deliberate variable privatization by ChatGPT led to incorrect results.

HotSpot

ChatGPT successfully parallelized the Hotspot application, recognizing two nested loops to optimize execution. One issue emerged when it was provided with a partial context (*omp_gpt* version) that omitted variable declarations. In these instances, ChatGPT independently included declarations that were incorrect and could lead to errors if the generated code integrated directly without modifications. In the initial attempts, both *omp_gpt* and *omp_gpt2* versions included a `collapse(2)` directive. While the *omp_gpt* version promptly removed `collapse(2)` after the first prompt, *omp_gpt2* required two very similar prompts to achieve the same result. GitHub Copilot faced a

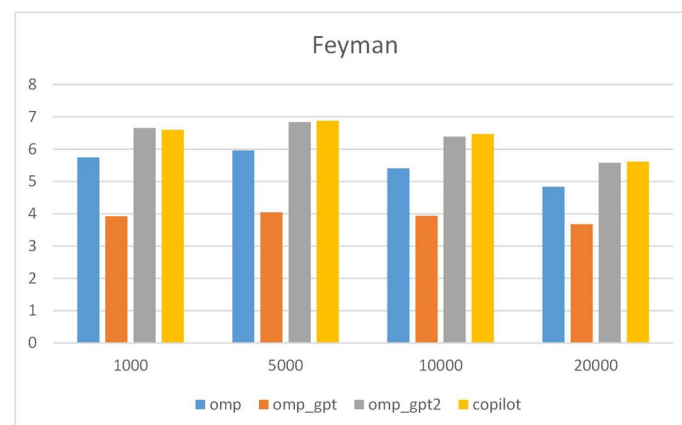


Fig. 2 Comparative Speedup Analysis of Feynman Algorithm Parallelization Using OpenMP with Different Approaches: *omp* (manual optimization), *omp_gpt* (ChatGPT's parallelization of a provided code snippet), *omp_gpt2* (ChatGPT's parallelization with full application context), and *copilot* (GitHub Copilot parallelization)

similar challenge with `collapse(2)`, maintaining it until it was explicitly commented out with `// no collapse`, after which it ceased to include the directive.

Furthermore, ChatGPT proposed that parallelization using `#pragma omp for` was more effective than utilizing tasks for this application. Since GitHub Copilot is limited to generating single blocks of code at a time, neither tool was able to fully optimize the application using task-based parallelism. This limitation is a contributing factor to why the *omp* version, manual optimization with a comprehensive understanding of the application, outperformed the other three variants for larger inputs, as can be seen at Fig. 3

Mandelbrot

In the baseline solution, calculation of the area of the Mandelbrot set is performed through an iterative, compute-bound algorithm with significant disbalances of the loop iterations. In parallel solution, to ensure correct calculations of the total number of points outside the set, a reduction was needed. The manually produced *omp* version considered those two problems and included `reduction` clause and hand-tuned `schedule(dynamic)` clause.

ChatGPT had problems with variable scoping and declaration in *omp_gpt* version when just a snippet of code is given, without variable declaration. The code did not compile, as some of the variables were declared as structures, rather than scalar variables. In the next step, the corrected code contained both `reduction` clause and `#pragma omp atomic` directive over a reduction variable that calculates the total number of points outside the set which was superfluous. When trying to optimize the code, `collapse` clause was added which resulted in a slowdown of the code. Only when loop iteration disbalance was suggested in the prompt, ChatGPT proposed to use `schedule(dynamic)` clause which brought performance similar to the manually produced *omp* version. In contrast, *omp_gpt2* version was produced with much less problem, due to the context available from the whole code. However, it retained a `collapse(2)` directive, which slightly slowed down the performance as depicted in Fig. 4. GitHub Copilot effectively parallelized the application from the beginning, identifying

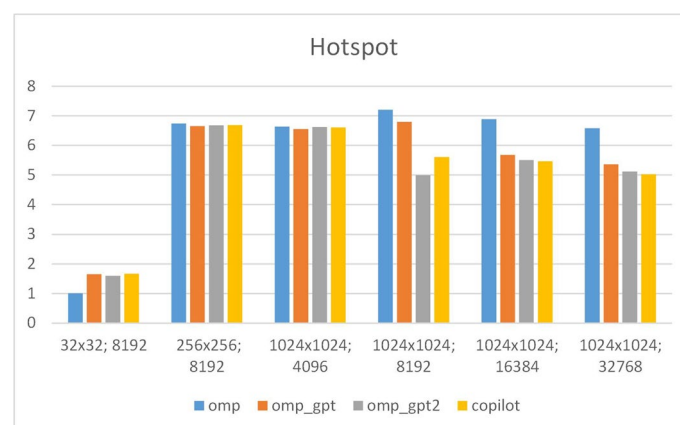


Fig. 3 Comparative Speedup Analysis of Hotspot Algorithm Parallelization Using OpenMP with Different Approaches: *omp* (manual optimization), *omp_gpt* (ChatGPT's parallelization of a provided code snippet), *omp_gpt2* (ChatGPT's parallelization with full application context), and *copilot* (GitHub Copilot parallelization)

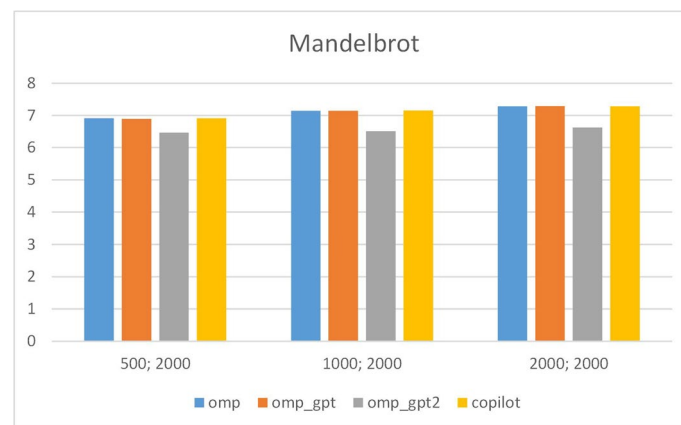


Fig. 4 Comparative Speedup Analysis of Mandelbrot Algorithm Parallelization Using OpenMP with Different Approaches: *omp* (manual optimization), *omp_gpt* (ChatGPT's parallelization of a provided code snippet), *omp_gpt2* (ChatGPT's parallelization with full application context), and *copilot* (GitHub Copilot parallelization)

the iteration imbalance and incorporating the `schedule(dynamic)` directive in the initial prompt.

MolDyn

Producing a parallel version of the MolDyn application was a challenging task both for manual parallelization and ChatGPT. The application itself consists of around 500 lines of code with different functions and loops suitable for parallelization. However, applying parallel directives to some of them incurred parallel overheads which reduced the maximum possible speedup when only hotspots in the code were annotated. In that context, the most time-consuming part of the code is the calculation of forces. Manually produced version primarily focused on that part of the code, where it was important to introduce `reduction` clauses for kinetic and potential energy calculation and synchronization on forces calculation. Other parts were also explored, as well as the strategies to tackle disbalances and thread reuse.

ChatGPT had problems producing both *omp_gpt* and *omp_gpt2* versions of the code. In the case of the former, synchronization constructs were omitted to protect array used in forces calculation, which was resolved in the next step using `#pragma omp atomic` directive which is appropriate for that task. However, a lot of problems arose in the case of the latter, *omp_gpt2* version, where the whole code was given. The produced code was overwhelmed with compilation errors related to variable scoping. It needed two more steps to resolve this matter, albeit the produced code did not produce the same results as the baseline version. Even further suggestions to protect reduction variables and shared arrays did not yield correct results, as compilation errors arose again. Finally, it produced the code which ran indefinitely. Thus, we reverted to the baseline version, and asked ChatGPT to produce and optimize just function that calculates forces in the baseline version. The correct version was produced, although performances were not as good as in *omp* and *omp_gpt* versions. It can be concluded that in this case, giving ChatGPT more context was not the best approach, as it confused the tool.

GitHub Copilot, on the other hand, displayed an ability to recognize the appropriate contexts for parallelization directives. Only with given # as a prompt it accurately inserted `#pragma omp for` or `#pragma omp atomic` directives. Initially, it omitted any directive for scheduling and then incorrectly added `schedule(static)` after the second prompt, which resulted in a suboptimal performance. Only after an explicit suggestion to use `schedule(dynamic)`, the Copilot incorporate it, leading to performance parity with the *omp* and *omp_gpt* cases. The performance evaluation for all variants can be observed in Fig. 5.

Nbody

Parallelization of the N-body simulation happened to be a challenging task in both *omp_gpt* and *omp_gpt2* variants. In *omp_gpt* variant ChatGPT initially introduced some non-existent private variables and later removed every private or shared directive, leading to inaccuracies in the results. The following attempts to tune the parallelization by addressing all four identified parallelization opportunities at once were unsuccessful, with only one section being correctly optimized. Subsequent efforts focused on individually tuning each of the remaining three sections led to successful and efficient parallelization.

In the *omp_gpt2* variant, ChatGPT had a similar issue with private and shared clauses. Additional effort was required to avoid persistently suggested `#pragma omp atomic` directives, which, while not violating the correctness, introduce unnecessary synchronization that degrades performance. The situation with Github Copilot was not much easier. With the initial, #, prompt a reduction clause on non-existent variable is introduced. Interestingly, this unconventional step did not prevent the code from running correctly, yet it negatively impacted the performance. It required once prompted to remove the spurious reduction and an additional one to employ the `collapse(2)` directive for loop nest optimizations.

The results, presented in Fig. 6, show performance for all variants, which can be attributed to the intrinsic complexity of the algorithm itself, which necessitates the completion of one iteration in its entirety before embarking on the next.

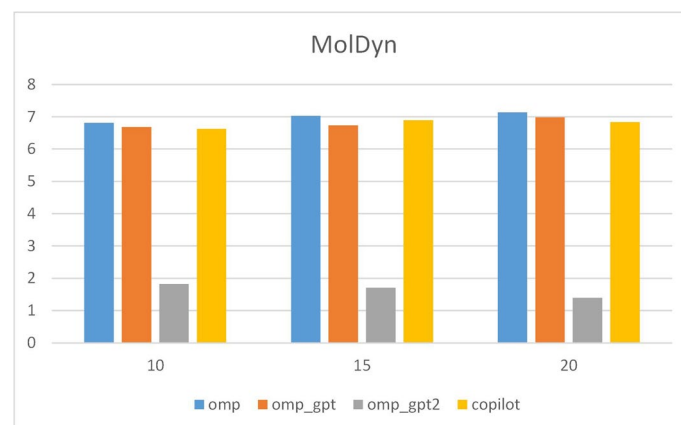


Fig. 5 Comparative Speedup Analysis of MolDyn Algorithm Parallelization Using OpenMP with Different Approaches: *omp* (manual optimization), *omp_gpt* (ChatGPT's parallelization of a provided code snippet), *omp_gpt2* (ChatGPT's parallelization with full application context), and *copilot* (GitHub Copilot parallelization)

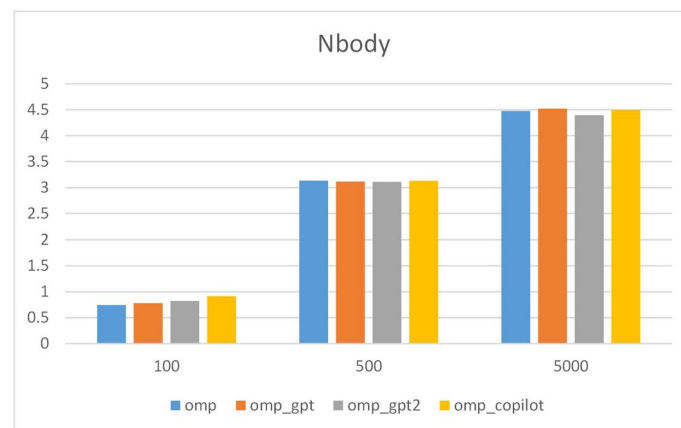


Fig. 6 Comparative Speedup Analysis of Nbody Algorithm Parallelization Using OpenMP with Different Approaches: *omp* (manual optimization), *omp_gpt* (ChatGPT's parallelization of a provided code snippet), *omp_gpt2* (ChatGPT's parallelization with full application context), and *copilot* (GitHub Copilot parallelization)

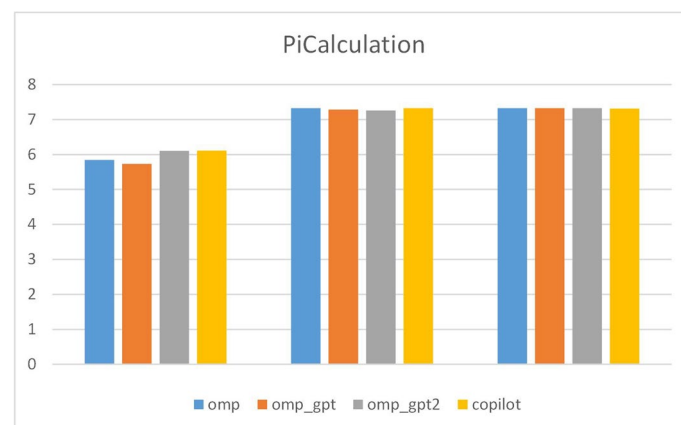


Fig. 7 Comparative Speedup Analysis of PiCalculation Algorithm Parallelization Using OpenMP with Different Approaches: *omp* (manual optimization), *omp_gpt* (ChatGPT's parallelization of a provided code snippet), *omp_gpt2* (ChatGPT's parallelization with full application context), and *copilot* (GitHub Copilot parallelization)

PiCalculation

Parallel solutions to this do not differ in *omp*, *omp_gpt*, *omp_gpt2*, and *copilot* versions, as can be seen in the Fig. 7. In all cases, ChatGPT correctly annotates the main for loop of the algorithm and declares the reduction variable. It is interesting that ChatGPT tries to optimize the code with the privatization of loop variables, which in this case significantly increases the speedup. However, it is not an OpenMP optimization and changes the original structure of the code, so we discarded it. With comparable precision, GitHub Copilot adeptly identified the outer loop as a prime candidate for parallelization, applying the necessary `reduction` clause to the sum variable to ensure the correctness and efficiency.

Prime

The main source of parallelism is the function that tests all the numbers in the range to test if they are prime numbers. It should be noted that it uses a rather inefficient algorithm to check whether a number is a prime number by checking if the number is divisible by any of the numbers between 2 and the number itself. The iterations of the outer loop are independent and can be easily distributed across threads. However, there is a significant work disbalance over different iterations, both because every second number is even (thus not prime) and larger numbers require more inner loop iterations to check if they are primes within the inner loop.

Manual OpenMP parallelization considered those problems. OpenMP `#pragma omp for` directive was used, but it was also decorated with `schedule(guided, 2)` clause to efficiently schedule loop iterations, as with guided schedule the chunk size starts as large and gradually decreases till given value, in order to progressively improve load balancing.

In the first attempt, *omp_gpt* version was correct. It uses OpenMP `#pragma omp for` directive with `reduction` clause properly placed, but not considered load balancing. When asked to optimize the code using OpenMP clauses, ChatGPT went the other direction and suggested a more efficient algorithm to check if the number was prime, by reducing the number of iterations of the inner loop. Also, it tried to privatize some iterator variables by changing their scope to improve performance. Only with the next explicit query to use `schedule` clause, it suggested using `schedule(dynamic)` to try to reduce disbalances.

The optimization step was more difficult in the case of *omp_gpt2* version. ChatGPT tried to optimize the whole code in a totally wrong direction by trying to parallelize the function used for testing leading to several versions of the code that do not compile or change the logic of the test function. In this case, having the whole code confused the tool and much more intervention from the user was needed to obtain correct parallel code. When it comes to scheduling directives, *omp_gpt2* provided a bit unusual `schedule(dynamic, 1000)` clause, with incorrect explanation why it can equally address work imbalances as efficiently as the `schedule(guided, 2)` clause.

Burdenless by the complexities that affected both ChatGPT variants, Github Copilot adeptly executed the parallelization of the outer loop. Notably, it not only identified the loop as a candidate for parallelization, but also applied the `schedule(guided, 2)` clause. This choice effectively addressed the inherent workload imbalance of the prime number testing algorithm. Owing to Copilot's operational constraints that prevent alteration of existing code and restrict its function to appending new blocks, it did not attempt to refine the underlying prime testing algorithm. Instead, it focused on optimizing the current method for parallel execution.

Figure 8 presents a comparison of performance outcomes across different versions. It indicates that the *omp*, *omp_gpt*, and *copilot* versions achieve comparable levels of performance. Meanwhile, the *omp_gpt2* variant shows a slight performance disadvantage, demonstrating lower understanding the nature of disbalance over iterations, even when full context is provided.

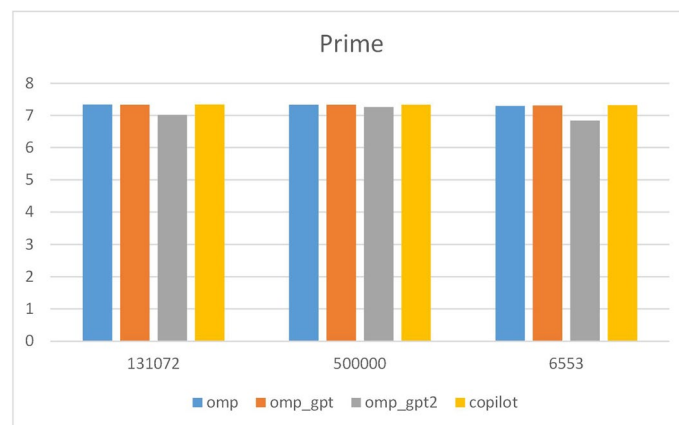


Fig. 8 Comparative Speedup Analysis of Prime Algorithm Parallelization Using OpenMP with Different Approaches: *omp* (manual optimization), *omp_gpt* (ChatGPT's parallelization of a provided code snippet), *omp_gpt2* (ChatGPT's parallelization with full application context), and *copilot* (GitHub Copilot parallelization)

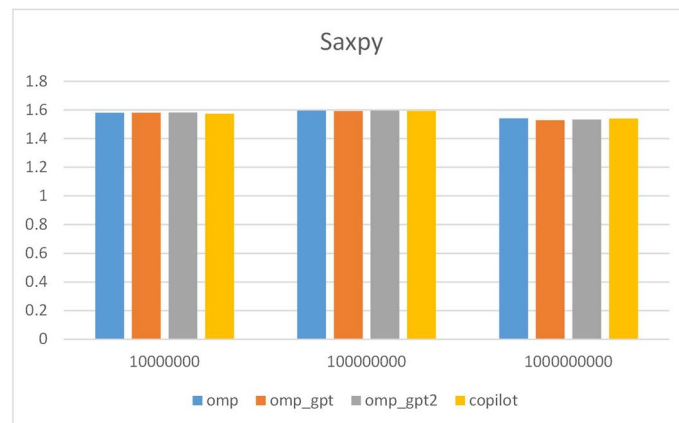


Fig. 9 Comparative Speedup Analysis of Saxpy Algorithm Parallelization Using OpenMP with Different Approaches: *omp* (manual optimization), *omp_gpt* (ChatGPT's parallelization of a provided code snippet), *omp_gpt2* (ChatGPT's parallelization with full application context), and *copilot* (GitHub Copilot parallelization)

Saxpy

Scalar multiplication and vector addition operation code represents rather simple processing. When asked to parallelize, ChatGPT suggested that each iteration of the loop depends on the result of the previous one, which was not true. However, the produced parallel code from both ChatGPT and Github Copilot was correct and it did not comprise unnecessary synchronization constructs. Figure 9 illustrates very similar achievements between variants, however all of them are limited to approximately 2, but anticipated to be closer to the number of cores. This can be primarily attributed to the constrained number of iterations, a result of the maximal array size that can be processed before encountering a segmentation fault. This size limitation results in a diminished opportunity to fully exploit the parallelism typically available in this computation. Also, this application is memory bound, thus dependant on the slow memory reads.

Sgemm

In a single precision dense matrix-matrix multiplication kernel, there are three nested loops that perform the operation. The first two loops are tightly nested which means that the OpenMP `collapse` clause can be applied to the code. It was manually applied in *omp* version of the code. It was also correctly recognized in both *omp_gpt* and *omp_gpt2* versions. It should be noted that ChatGPT also introduced `#pragma omp atomic` directive in *omp_gpt* version to protect the writes to each element of the resulting matrix. At first, the code did not compile, as it was applied in a wrong way, as arithmetic operations inside atomic directive are not allowed. It was corrected with an introduction of an intermediate variable. However, it was superfluous and introduced unnecessary overheads, as each element was processed by just one thread. Also, we noticed the unnecessary use of `firstprivate` clause in *omp_gpt2* version for several shared variables. Although it might represent an attempt to privatize them by replication, ChatGPT did not suggest that in the prompt, and it also did not contribute to the performance of the code.

GitHub Copilot successfully inserted `#pragma omp parallel` for directive with appropriate distinction between private and shared variables. Nevertheless, Copilot initially did not apply the `collapse(2)` clause that could optimize the tightly nested loops. However, with a second prompt, it correctly integrated the clause, aligning with the optimal parallelization strategy previously applied manually and by ChatGPT in the both *omp_gpt* and *omp_gpt2* versions. The performance for each parallelization approach are presented in Fig. 10.

Discussion

While conducting the experiments, we have observed several trends in using LLMs for OpenMP-based parallelization. Generally, LLM-based tools can be valuable support to the programmer in writing parallel programs based on shared memory programming model, such as OpenMP. Such tools can successfully analyze the code and annotate its parts for parallel execution. Still, it is crucial for programmers to not only

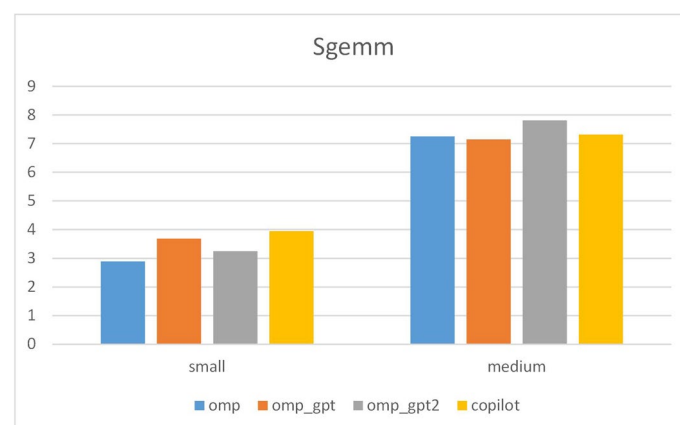


Fig. 10 Comparative Speedup Analysis of Sgemm Algorithm Parallelization Using OpenMP with Different Approaches: *omp* (manual optimization), *omp_gpt* (ChatGPT's parallelization of a provided code snippet), *omp_gpt2* (ChatGPT's parallelization with full application context), and *copilot* (GitHub Copilot parallelization)

rely on the correctness and speedup achieved by the LLM's output but also to utilize their own expertise in OpenMP to recognize further optimization opportunities. Even when the code generated by these tools is functionally correct and offers some speedup, a skilled programmer must discern, if any, areas for enhancement and guide the tool how to achieve better performance. However, following guidelines should be followed in order to avoid typical mistakes in their application:

- Both ChatGPT and Github Copilot exhibited problems with scoping of variables, as they try both to move variable declarations to the inner blocks of code or incorrectly use OpenMP scoping clauses, such as `shared`, `private`, and `firstprivate`. It is more pronounced in the case of ChatGPT, where such problems often lead to compilation and runtime errors, especially with deliberate variable privatization seen in some cases. The problem is more exaggerated in more complex codes.
- Produced code should be checked for correct synchronization over shared objects. Reduction variables are usually correctly recognized by both tools. However, in some cases, such as MolDyn example, ChatGPT omitted synchronization directives over shared arrays. In even more cases, unnecessary synchronization using `#pragma omp atomic` directive (Sgemm example) and superfluous synchronization using combination of `#pragma omp atomic` directive and `reduction` clause was used (Mandelbrot example). While generally it does not harm correctness, it can negatively impact performance due to overheads. Even more attention should be paid with Github Copilot, as it produces code only for the local scope. Synchronization directives should be inserted with additional prompts at the manually defined place in code.
- ChatGPT and Github Copilot had problems to properly use `collapse` clause. In several cases, both tools insisted on using `collapse` clause, although it led to decrease in performance. On the other hand, ChatGPT was able to reorganize the loops in order to apply such clause, as was the case in Feynman example.
- In many cases, optimal performance can be only achieved with appropriate workload balancing among the parallel threads. Both tools need some guidance through prompts, as they are not able to detect loop iteration imbalances in all such cases. That is also the case with chunk size parameter of the `schedule` clause.

As mentioned before, Github Copilot always analyzes the full context of the source code and generates code on demand [63]. However, it produces code for the local context. In case of OpenMP directives, that means the programmer should point out to the each place a directive or clause should be inserted with appropriate prompt. In that sense, Github Copilot is more labor-intensive to the programmer comparing to the ChatGPT. Generally, it is in accordance with some other studies which evaluated Github Copilot [64, 65]. Those studies suggest that Github Copilot is essentially used either to produce code faster or to explore the possible options for the programmer [65]. However, due to potential problems in the code, it is more useful for expert programmers [64]. Nevertheless, Github Copilot produced correct programs for all tested mini-applications in this study. This observation is aligned with the general

efficacy reported in the broader context of AI-assisted code generation, where tools like GitHub Copilot have been shown to generate code that compiles and executes correctly across various coding tasks [66–68].

ChatGPT can reorganize the given piece of code in more comprehensive fashion. Our experiments showed that it was able to restructure the code in several cases to utilize OpenMP directives and clauses more efficiently. It was especially pronounced in the case of `collapse` clause. Moreover, ChatGPT often performed privatization of variables by declaring them in inner scopes. This can be seen as a non-OpenMP performance optimization, which led to performance improvements on several occasions.

Generally, ChatGPT requires less effort from the programmer than Github Copilot. However, sometimes it can be confused when the large code context is given, often leading to compilation errors, erroneous execution, or even significant changes of the original code, which is noted in open literature for other programming tasks [69]. Our experience is that it is best to provide ChatGPT with code context on the function level, as it often includes necessary variable declarations and scoping, but limits parallelization efforts to the processing performed in the function itself. If available to the programmer, both tools can be used in synergistic fashion: ChatGPT can be used to point out to all locations in code that need parallelization directives and clauses, while Github Copilot can be used to correctly include them with several shorter prompts.

In the similar experiments with ChatGPT and OpenACC [48], the authors noted ChatGPT can be utilized to generate code without serious issues efficiently using OpenACC directives. However, manual changes in the code are needed to improve its performance further. Our experiments showed that performance can be improved with ChatGPT with appropriate prompt engineering as well.

Conclusion

In this paper, we examined the potential of LLMs for OpenMP-based parallelization with the focus on ChatGPT and Github Copilot tools. We conducted a series of experiments using nine different real-life mini-applications in which we covered different parallelization approaches. The results showed promising capabilities of LLMs, although ChatGPT was not always able to produce correct parallel code. That conclusion is even more emphasized in the case of performance optimizations. Regarding the performance, obtained speedups of the OpenMP solutions produced by ChatGPT were in most cases similar or slightly below to those manually produced. However, obtained speedups were lower in the cases of more complex programs where significantly more effort from the programmer is needed to obtain good results. On the other hand, Github Copilot showed significantly more mature behavior, matching the performance of manually written code in most cases.

There are several directions for future work. There are OpenMP constructs, such as *task* directive and thread affinity clauses, that can be further investigated with LLM-based tools. Similar experiment can be conducted using other popular parallel programming models, such as Pthreads, MPI, CUDA, OpenCL, OpenACC, Kokkos, and Raja with an emphasis on GPU implementations, as well as more advanced features introduced from OpenMP 4.0 specification targeting accelerators. Programming models, such as MPI and CUDA, require significantly more code restructuring of sequential

code, and it would be beneficial to test LLMs in such a setting. Also, future studies can focus on complete implementations of the popular benchmark suites, such as Rodinia, Parboil, NAS, and PolyBench. Our benchmark suite can be expanded with more mini-applications from the computer science domain, such as graph traversals, dynamic programming, back-tracking algorithms and similar.

The other promising direction is the evaluation of state-of-the-art LLMs trained specifically on code and HPC workloads, such as OMPify, AutoParLLM, and OMPGPT, using aforementioned benchmark suites which contain mini-applications that represent real-life workloads. Conclusions and insights of such studies can be used in turn to train even more efficient LLMs for HPC purposes.

Supplementary Information

The online version contains supplementary material available at <https://doi.org/10.1186/s40537-024-01019-z>.

Supplementary Material 1.

Acknowledgements

Not applicable.

Author contributions

Conceptualization, Marko Mišić; methodology, Marko Mišić; software, Marko Mišić and Matija Dodović; validation, Marko Mišić and Matija Dodović; formal analysis, Marko Mišić; investigation, Marko Mišić and Matija Dodović; data curation, Marko Mišić and Matija Dodović; writing—original draft preparation, Marko Mišić and Matija Dodović; writing—review and editing, Marko Mišić and Matija Dodović; visualization, Matija Dodović; supervision, Marko Mišić; project administration, Marko Mišić; funding acquisition, Marko Mišić. All authors have read and approved the final manuscript.

Funding

This work was financially supported by the Ministry of Science, Technological Development and Innovation of the Republic of Serbia under contract number: 451-03-65/2024-03/200103.

Availability of data and materials

No datasets were generated or analysed during the current study.

Code availability

Implementations are available on the following link: <https://github.com/mdodovic/OMP-ChatGPT-and-Copilot>.

Declaration

Ethics approval and consent to participate

Not applicable.

Consent for publication

All authors have read and approved the final manuscript.

Competing interests

The authors declare that they have no competing interests.

Received: 3 May 2024 Accepted: 13 October 2024

Published online: 10 November 2024

References

1. Amber Q, Hashmi FA. ChatGPT revolution in different fields of life: literature synthesis and research agenda. 2023.
2. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al. Attention is all you need. *Adv Neural Inform Process Syst*. 2017;30.
3. Reisenbichler M, Reutterer T, Schweidel DA, Dan D. Frontiers: supporting content marketing with natural language generation. *Market Sci*. 2022;41(3):441–52.
4. Wu S, Irsoy O, Lu S, Dabrowski V, Dredze M, Gehrmann S, et al. Bloomberggpt: a large language model for finance. *arXiv preprint arXiv:2303.17564*. 2023.
5. Oliński M, Krukowski K, Sieciński K. Bibliometric overview of ChatGPT: new perspectives in social sciences. *Publications*. 2024;12(1):9.

6. Yang X, Chen A, PourNejatian N, Shin HC, Smith KE, Parisien C, et al. A large language model for electronic health records. *NPJ Digit Med*. 2022;5(1):194.
7. Zhang S, Fan R, Liu Y, Chen S, Liu Q, Zeng W. Applications of transformer-based language models in bioinformatics: a survey. *Bioinform Adv*. 2023;3(1): vbad001.
8. He Z, Liang T, Jiao W, Zhang Z, Yang Y, Wang R, et al. Exploring human-like translation strategy with large language models. *Trans Assoc Comput Linguist*. 2024;12:229–46.
9. Kasneci E, Seßler K, Küchemann S, Bannert M, Dementieva D, Fischer F, et al. ChatGPT for good? On opportunities and challenges of large language models for education. *Learn Individ Differ*. 2023;103: 102274.
10. Yang J, Jin H, Tang R, Han X, Feng Q, Jiang H, et al. Harnessing the power of LLMs in practice: a survey on ChatGPT and beyond. *ACM Trans Knowl Discov Data*. 2023. <https://doi.org/10.1145/3649506>.
11. Hadi MU, Qureshi R, Shah A, Irfan M, Zafar A, Shaikh MB, et al. Large language models: a comprehensive survey of its applications, challenges, limitations, and future prospects. *Authorea Preprints*. 2023.
12. Chen M, Tworek J, Jun H, Yuan Q, Pinto HPdO, Kaplan J, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*. 2021.
13. Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, et al. Language models are few-shot learners. *Adv Neural Inform Process Syst*. 2020;33:1877–901.
14. Li Y, Choi D, Chung J, Kushman N, Schrittwieser J, Leblond R, et al. Competition-level code generation with alphacode. *Science*. 2022;378(6624):1092–7.
15. Roziere B, Gehring J, Gloeckle F, Sootla S, Gat I, Tan XE, et al. Code llama: open foundation models for code. *arXiv preprint arXiv:2308.12950*. 2023.
16. Nam D, Macvean A, Hellendoorn V, Vasilescu B, Myers B. Using an LLM to help with code understanding. In: 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE). IEEE Computer Society; 2024. p. 881.
17. Lu J, Yu L, Li X, Yang L, Zuo C, LLaMA-reviewer: advancing code review automation with large language models through parameter-efficient fine-tuning. In: IEEE 34th International Symposium on Software Reliability Engineering (ISSRE). vol. 2023. IEEE; 2023. pp. 647–58.
18. Ishizue R, Sakamoto K, Washizaki H, Fukazawa Y. Improved program repair methods using refactoring with GPT models. In: Proceedings of the 55th ACM Technical Symposium on Computer Science Education. Vol. 1. 2024; pp. 569–75.
19. Nichols D, Davis JH, Xie Z, Rajaram A, Bhatele A. Can large language models write parallel code? *arXiv preprint arXiv:2401.12554*. 2024.
20. Vaithilingam P, Zhang T, Glassman EL. Expectation vs. experience: evaluating the usability of code generation tools powered by large language models. In: Chi Conference on Human Factors in Computing Systems Extended Abstracts; 2022. pp. 1–7.
21. Wermelinger M. Using github copilot to solve simple programming problems. In: Proceedings of the 54th ACM Technical Symposium on Computer Science Education Vol. 1; 2023. pp. 172–8.
22. Lau S, Guo P. From “Ban it till we understand it” to “Resistance is futile”: how university programming instructors plan to adapt as more students use AI code generation and explanation tools such as ChatGPT and GitHub Copilot. In: Proceedings of the 2023 ACM Conference on International Computing Education Research-Vol. 1; 2023. pp. 106–21.
23. Banković M, Filipović V, Graovac J, Hadži-Purić J, Hurson AR, Kartelj A, et al. Teaching graduate students how to review research articles and respond to reviewer comments. In: Hurson AR, Milutinović V, editors., et al., *Advances in computers*, vol. 116. Amsterdam: Elsevier; 2020. p. 1–63.
24. Robey R, Zamora Y. Parallel and high performance computing. New York: Simon and Schuster; 2021.
25. Yin F, Shi F. A comparative survey of big data computing and HPC: From a parallel programming model to a cluster architecture. *Int J Parallel Progr*. 2022;50(1):27–64.
26. Babović Z, Bajat B, Đokić V, Đorđević F, Drašković D, Filipović N, et al. Research in computing-intensive simulations for nature-oriented civil-engineering and related scientific fields, using machine learning and big data: an overview of open problems. *J Big Data*. 2023;10(1):73.
27. Babović Z, Bajat B, Barac D, Bengin V, Đokić V, Đorđević F, et al. Teaching computing for complex problems in civil engineering and geosciences using big data and machine learning: synergizing four different computing paradigms and four different management domains. *J Big Data*. 2023;10(1):89.
28. Czarnul P, Prończak J, Drypczewski K. Survey of methodologies, approaches, and challenges in parallel programming using high-performance computing systems. *Sci Progr*. 2020;2020:1–19.
29. de Supinski BR, Scogland TR, Duran A, Klemm M, Bellido SM, Olivier SL, et al. The ongoing evolution of OpenMP. *Proc IEEE*. 2018;106(11):2004–19.
30. Arabnejad H, Bispo J, Cardoso JM, Barbosa JG. Source-to-source compilation targeting OpenMP-based automatic parallelization of C applications. *J Supercomput*. 2020;76(9):6753–85.
31. Hoffmann RB, Löff J, Griebler D, Fernandes LG. OpenMP as runtime for providing high-level stream parallelism on multi-cores. *J Supercomput*. 2022;78(6):7655–76.
32. Mahmud QI, TehraniJamsaz A, Phan HD, Ahmed NK, Jannesari A. Autoparllm: Gnn-guided automatic code parallelization using large language models. *arXiv preprint arXiv:2310.04047*. 2023.
33. Kadosh T, Schneider N, Hasabnis N, Mattson T, Pinter Y, Oren G. Advising OpenMP parallelization via a graph-based approach with transformers. In: McIntosh-Smith S, Klemm M, de Supinski BR, Deakin T, Klinkenberg J, editors. *International Workshop on OpenMP*. Berlin: Springer; 2023. p. 3–17.
34. Chen L, Bhattacharjee A, Ahmed N, Hasabnis N, Oren G, Vo V, et al. OMPGPT: a generative pre-trained transformer model for OpenMP. *arXiv preprint arXiv:2401.16445*. 2024.
35. Chen L, Ding X, Emani M, Vanderbruggen T, Lin PH, Liao C. Data race detection using large language models. In: Proceedings of the SC’23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis; 2023. pp. 215–23.
36. Ding X, Chen L, Emani M, Liao C, Lin PH, Vanderbruggen T, et al. Hpc-gpt: integrating large language model for high-performance computing. In: Proceedings of the SC’23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis; 2023. pp. 951–60.

37. Chen L, Lin PH, Vanderbruggen T, Liao C, Emani M, de Supinski B. Lm4hpc: towards effective language model application in high-performance computing. In: McIntosh-Smith S, Klemm M, de Supinski BR, Deakin T, Klinkenberg J, editors. *International Workshop on OpenMP*. Berlin: Springer; 2023. p. 18–33.
38. Blagojević V, Bojić D, Bojović M, Cvetanović M, Đorđević J, Đurđević Đ, et al. A systematic approach to generation of new ideas for PhD research in computing. In: Hurson AR, Milutinović V, editors., et al., *Advances in computers*, vol. 104. Amsterdam: Elsevier; 2017. p. 1–31.
39. Aragon R. *An introduction to parallel programming*; 2023.
40. Liu F, Li G, Zhao Y, Jin Z. Multi-task learning based pre-trained language model for code completion. In: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*; 2020. pp. 473–85.
41. Cruz-Benito J, Vishwakarma S, Martin-Fernandez F, Faro I. Automated source code generation and auto-completion using deep learning: comparing and discussing current language model-related approaches. *AI*. 2021;2(1):1–16.
42. Perez L, Ottens L, Viswanathan S. Automatic code generation using pre-trained language models. *arXiv preprint arXiv:2102.10535*. 2021.
43. Xu FF, Alon U, Neubig G, Hellendoorn VJ. A systematic evaluation of large language models of code. In: *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*; 2022. pp. 1–10.
44. Fajković E, Rundberg E. The impact of AI-generated code on web development: a comparative study of ChatGPT and GitHub Copilot. *Faculty of Engineering, Blekinge Institute of Technology, Karlskrona, Sweden*.
45. Nichols D, Marathe A, Menon H, Gamblin T, Bhatele A. Modeling parallel programs using large language models. *arXiv preprint arXiv:2306.17281*. 2023.
46. Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, et al. Chain-of-thought prompting elicits reasoning in large language models. *Adv Neural Inform Process Syst*. 2022;35:24824–37.
47. Kadosh T, Hasabnis N, Mattson T, Pinter Y, Oren G. Quantifying OpenMP: statistical insights into usage and adoption. In: *IEEE High Performance Extreme Computing Conference (HPEC)*. vol. 2023. IEEE; 2023. pp. 1–7.
48. Costa E, Silva G. Evaluating the use of ChatGPT for parallel programming with OpenACC. In: *Proceedings of 39th International Conference on Computers and Their Applications*. vol. 98. 2024. pp. 1–14.
49. v Kistowski J, Arnold JA, Huppler K, Lange KD, Henning JL, Cao P. How to build a benchmark. In: *Proceedings of the 6th ACM/SPEC international conference on performance engineering*; 2015. pp. 333–6.
50. Bailey D, Harris T, Saphir W, Van Der Wijngaart R, Woo A, Yarrow M. The NAS parallel benchmarks 2.0. Technical Report NAS-95-020, NASA Ames Research Center; 1995.
51. Karimov J, Rabl T, Markl V. Polybench: The first benchmark for polystores. In: *Performance Evaluation and Benchmarking for the Era of Artificial Intelligence: 10th TPC Technology Conference, TPCTC 2018, Rio de Janeiro, Brazil, August 27–31, 2018, Revised Selected Papers 10*. Springer; 2019. pp. 24–41.
52. Che S, Boyer M, Meng J, Tarjan D, Sheaffer JW, Lee SH, Rodinia: A benchmark suite for heterogeneous computing. In, et al. *IEEE International Symposium on Workload Characterization (IISWC)*. vol. 2009. IEEE. 2009. pp. 44–54.
53. Stratton JA, Rodrigues C, Sung IJ, Obeid N, Chang LW, Ansari N, et al. Parboil: a revised benchmark suite for scientific and commercial throughput computing. *Center for Reliable and High-Performance Computing*. 2012;127(7.2).
54. de Araujo GA, Griebler D, Danelutto M, Efficient Fernandes LG. NAS parallel benchmark kernels with CUDA. In: *28th Euromicro international conference on parallel, distributed and network-based processing (PDP)*. 2020. IEEE; 2020. pp. 9–16.
55. Đukić J, Mišić M. An evaluation of directive-based parallelization on the GPU using a parboil benchmark. *Electronics*. 2023;12(22):4555.
56. Burkardt J. John Burkardt's home page. <https://people.sc.fsu.edu/~jburkardt/>. Accessed 16 Apr 2024.
57. EPCC. Edinburgh Parallel Computing Centre Code Repository. <https://github.com/EPCCed>. Accessed 16 Apr 2024.
58. Pacheco P. *An introduction to parallel programming*. Amsterdam: Elsevier; 2011.
59. AMD. HPC training examples. <https://github.com/amd/HPCTrainingExamples/>. Accessed 16 Apr 2024.
60. Asanovic K, Bodik R, Catanzaro BC, Gebis JJ, Husbands P, Keutzer K, et al. *The landscape of parallel computing research: a view from Berkeley*. Berkeley: University of California; 2006.
61. Che S, Sheaffer JW, Boyer M, Szafaryn LG, Wang L, Skadron K. A characterization of the Rodinia benchmark suite with comparison to contemporary CMP workloads. In: *IEEE International Symposium on Workload Characterization (IISWC'10)*. IEEE; 2010. pp. 1–11.
62. Doerfert J, Finkel H. Compiler optimizations for OpenMP. In: *Evolving OpenMP for Evolving Architectures: 14th International Workshop on OpenMP, IWOMP 2018, Barcelona, Spain, September 26–28, 2018, Proceedings 14*. Springer; 2018. pp. 113–27.
63. Github. About GitHub Copilot Individual. <https://docs.github.com/en/copilot/copilot-individual/about-github-copilot-individual>. Accessed 16 Apr 2024.
64. Dakhel AM, Majdinasab V, Nikanjam A, Khomh F, Desmarais MC, Jiang ZMJ. Github copilot ai pair programmer: asset or liability? *J Syst Softw*. 2023;203: 111734.
65. Barke S, James MB, Polikarpova N. Grounded copilot: how programmers interact with code-generating models. *Proc ACM on Progr Lang*. 2023;7(OOPSLA1):85–111.
66. Peng S, Kalliamvakou E, Cihon P, Demirel M. The impact of AI on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590*. 2023.
67. Yetiştirilen B, Özsoy I, Ayerdem M, Tüzün E. Evaluating the code quality of AI-assisted code generation tools: an empirical study on github copilot, amazon codewhisperer, and chatgpt. *arXiv preprint arXiv:2304.10778*. 2023.
68. Wong D, Kothig A, Lam P. Exploring the verifiability of code generated by github copilot. *arXiv preprint arXiv:2209.01766*. 2022.
69. Liu J, Xia CS, Wang Y, Zhang L. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Adv Neural Inform Process Syst*. 2023;36.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.